



AUTOMATIC MACHINE LEARNING

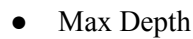
Tutorial 4 Report: Data Pre-Processing and Evaluation.

Jin Weijian Víctor Chen - 100496723

Teresa Vicente Sánchez - 100496795

1. Select a new set of parameters considering the mentioned attributes, and perform a training process with the addition of the cross validation. What are the Mean Squared Error (MSE), Root Mean Squared Error (RMSE), and Mean Absolute Error (MAE) of the models created? Use a table to compare these results. Interpret the results.

 grid_search_results_1 (1).xlsx



- Min Samples Split

The dot plot shows the distribution of metric sums for different minimum sample split sizes. The x-axis represents the 'Min Samples Split' (1, 10, 20, 30, 40) and the y-axis represents the 'Sum of Metrics' (0 to 6). For each split size, a blue vertical bar indicates the range of metric sums, with dots at the minimum and maximum values.

Min Samples Split	Min Sum of Metrics	Max Sum of Metrics
1	~3.8	~5.8
10	~3.8	~5.0
20	~3.7	~4.6
40	~3.7	~4.3



- Criterion

Criterion	Sum of Metrics
1	4.0
2	4.1
3	4.2
4	4.3
5	4.4
6	4.5
7	4.6
8	4.7
9	4.8
10	4.9
11	5.0
12	5.1
13	5.2
14	5.3
15	5.4
16	5.5
17	5.6
18	5.7
19	5.8
20	5.9



1

2. Select the three best models and perform the prediction of the test data. What are the Mean Squared Error (MSE), Root Mean Squared Error (RMSE), and Mean Absolute Error (MAE) of the models? Which model is better? Compare the models in a table.

The 3 best models taken from above are in the table below:

Max Depth	Min Samples Split	Max Features	Criterion	Splitter	MSE	RMSE	MAE
None	40	0,75	squared_error	random	1,474436574	1,214129546	0,9464918225
20	40	None	squared_error	random	1,497457491	1,223390168	0,9524020547
10	40	log2	friedman_mse	random	1,494595505	1,222159294	0,9572037126

In order to choose the best models we thought about two options, the first one was to use the sum of the metrics and choosing as the best models those with smaller metrics or by doing the same but ponderating each metric depending on its importance. We thought that the best option would be the first one as it was much simpler and the three metrics were equally as valuable (even though the RMSE is the root squared of the MSE, taking out this metric doesn't affect the results).

The prediction with the test data gives us the following metrics:

Model	MSE	RMSE	MAE
1	0.265063	0.514843	0.295325
2	0.239547	0.489436	0.253765
3	0.426386	0.652982	0.562707

From these numbers we can see that Model 2 is definitely the best choice of model. This Model has the lowest values for the three metrics we are using (MSE, RMSE and MAE). This suggests that Model 3 makes smaller errors on average (MSE), gets the closest predictions to the actual values (RMSE) and the average magnitude of errors is also the smallest (MAE). This is

why, in terms of the pendulum simulation, Model 2 which is more accurate and consistent should work better.

At the beginning, when writing this code we made the mistake of forgetting to add `np.random.seed(42)` which made every output we generated different. With this we have learnt that setting a fixed number as a seed in our model training is essential in order to ensure that the work is reproducible, consistent, and reliable.

3. Select the best model from the previous point and retrain the models with all the training data with no cross validation. Predict the test data again. Do the results differ from the model with cross validation? Quantify this difference.

By taking a simple look at the numbers, we can already realize that there is a substantial difference between the cross-referenced values and these.

Model	MSE	RMSE	MAE
2	0.7422813	0.8615575	0.7674360

$$\text{Percentage Difference} = \frac{\text{Difference}}{\text{Without Cross-Validation}} \cdot 100$$

Metric	Without Cross-validation	With Cross-validation	Difference	Percentage Difference
MSE	0,7423	0,2395	0,5028	67.75%
RMSE	0,8616	0,4894	0,3722	43.20%
MAE	0,7674	0,2538	0,5136	66.92%

For the **mean squared error (MSE)** we can see that the values drop from 0.7423 (without using cross-validation) to 0.2395 (using cross-validation), which results in a difference of 0.5028. This implies a 67.75% reduction in the MSE.

For the **root mean squared error (RMSE)** the value is decreased from 0.8616 to 0.4894 with cross validation. This results in a 43.20% reduction which is crucial as it means that cross validation makes the results more accurate and reliable.

For the **mean absolute error (MAE)**, the difference between the cross validated model and the non-cross validated model is 0.5136 (from 0.7674 to 0.2538), which gives a percentage difference of 66.92%.

From all of this we can conclude that cross validation is not only useful, but essential as it can generalize better to unseen data. The advantage of cross-validation is that, as it uses a lot of subsets of data, cross-validation helps prevent overfitting and makes sure there is a constant performance as can be seen from the quantified differences in this exercise.

Exercise 2: Linear Regression and SVM

1. Perform a linear regression with all the training data and use the test data to determine the Mean Squared Error (MSE), Root Mean Squared Error (RMSE), and Mean Absolute Error (MAE) of the model. Is the model good?

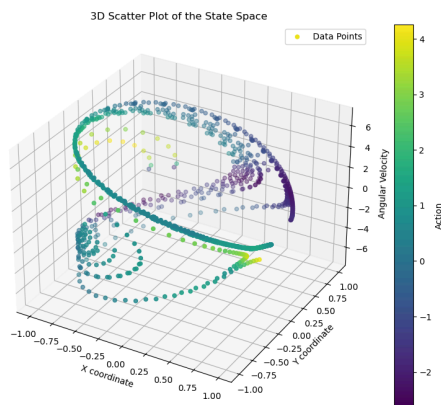
Mean Squared Error - 0.666: The MSE shows the average of the squares of the errors, which also is the average squared difference between the estimated values and the actual value. The lower the MSE value, the better fit of the model to the data. In comparison with the first exercise we can see that the values are lower, which does indicate that this would be a better model.

Root Mean Squared Error - 0.816: RMSE is the square root of MSE and provides a measure of the magnitude of the error in the same units as the target variable, which makes it a bit more interpretable. This value shows how much the predictions deviate from the actual values (on average). Again, in comparison with the first exercise the values are lower, making it a better model; but we still believe that the numbers are not low enough.

Mean Absolute Error - 0.705: MAE gives us the average of the absolute errors. This is the average over the test sample of the absolute differences between the prediction and the actual observation (and all individual differences have equal weight). This metric is the most useful because it gives a more direct interpretation of the average error magnitude without squaring, meaning that it does not penalize larger errors (when the MSE and RMS do).

By only using these metrics we can consider that the model is better than the first one, but we wouldn't say that it is directly "good", as, in comparison with the action values which range from X-X, the numbers are still quite high.

2. Visualize the state space using a 3d plot. Can you see why this model is not working well?



The main reason we believe that this model is not working correctly is that by applying a linear regression, we are assuming a linear relationship between the three features (x, y and angular velocity) and the target variable (action). And as we can see from the plot, it doesn't look linear at all.

Moreover, the distribution of the data points show very complex relationships that linear regression cannot capture. For instance, the color gradient (which represents 'Action') shows a complex pattern across the feature space which modeling with a linear regression is impossible.

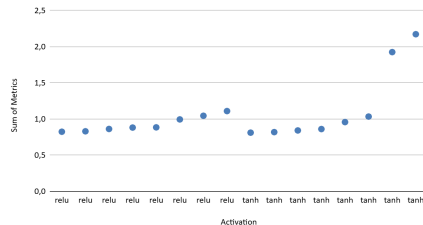
Exercise 3: Neural Networks

1. Select a new set of parameters considering the mentioned attributes, and perform a training process. What are the Mean Squared Error (MSE), Root Mean Squared Error (RMSE), and Mean Absolute Error (MAE) of the models created? Use a table to compare these results.

The results from this model can be checked in the following table

[X mlp_model_evaluation_results.xlsx](#)

Sum of Metrics frente a Activation



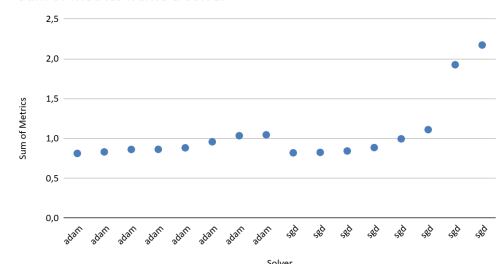
• Activation Function

The metric values of ReLU are consistently low across multiple trials, which shows that it performs reliably. On the other hand, Tanh has performed more or less the same except for some notable outliers (around 1.8 and 2.3), indicating a bit of variability that might occasionally lead to worse outcomes. For this reason, sticking with ReLU might be a better option most of the time.

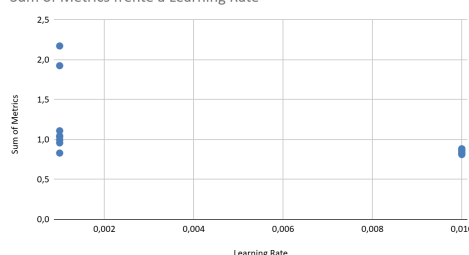
• Solver

Here we have a very similar pattern to that in the Activation Function. The Adam solver shows a gradual decrease in performance but the greatest sum of metrics it has is a bit higher than 1.0. In comparison, the SGD performs in a similar way at the beginning but has a few outliers at the end. This indicates that the Adam solver is more useful as it, generally speaking, leads to lower metrics.

Sum of Metrics frente a Solver



Sum of Metrics frente a Learning Rate



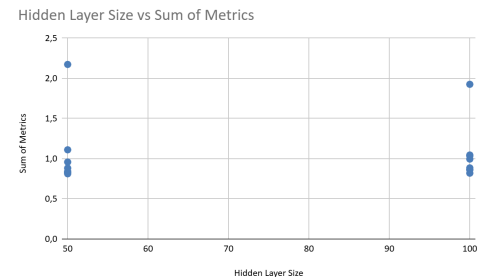
• Learning Rate

It's quite evident from the dispersion graph that a learning rate of 0.01 is ideal. For the 0.01 rate, the points cluster closer together around the lower sum of metrics range (0.8-0.9), demonstrating consistent and low error. In contrast, at 0.001, the data points are more spread out, indicating higher variability in both lower and higher values of the sum of metrics. Clearly, 0.01 proves to be much better because it not only

shows tighter clustering of performance metrics but also suggests a more reliable and stable model behavior, making it the superior choice for optimal results.

- Hidden Size Layer

Model performance is quite similar across different hidden layer sizes. Specifically, at both 50 and 100 hidden layers, the majority of data points cluster around a sum of metrics value of 1.0. Each size also displays an outlier near 2.0, with the outlier for 50 hidden layers being slightly higher, though this difference isn't substantial enough to impact overall performance conclusions. This indicates that both hidden layer sizes perform in a very similar manner, suggesting flexibility in choosing either size without any significant loss.



2. Select the best model from the previous point and retrain the models with early stopping set to False. Predict the test data again. Do the results differ from the model with cross validation? Quantify this difference.

The best model out of all of the above is, and the values with the early stopping set to TRUE is :

- Activation Function: Tanh
- Solver: Adam
- Hidden Layer Sizes: (50,)
- Learning Rate Initialization: 0.01

We chose the best model in the same way as we did the rest. As we thought that all of the metrics had the same importance we added them up in another column and found the lowest values

Here are the values with the early-stopping set to TRUE and set to FALSE:

early_stopping=True		
MSE	RMSE	MAE
0,185328784	0,430498297	0,196212468

early_stopping=False		
MSE	RMSE	MAE
0,195203008	0,441817845	0,335570431

Early stopping in the MLP Model is a technique used to prevent overfitting during the training. This ensures that the model does not overlearn, because this would include noise and random details instead of focusing on the underlying patterns, which leads to worse performance on new data.

The differences between the early stopping being set to True or False is that when it is set to True the training stops early if the performance stops improving, even if the maximum iterations haven't been reached and it also saves a small part of the data as the validation set. While if it is set to False the model always trains for all the iterations and no validation set is used.

In order to quantify this data we used the difference and the percentage difference which are defined as:

$$\text{Difference} = \text{Early stopping (False)} - \text{Early stopping (True)} \text{ and the}$$

$$\text{Percentage Difference} = \frac{\text{Difference}}{\text{Early Stopping (True)}} \cdot 100$$

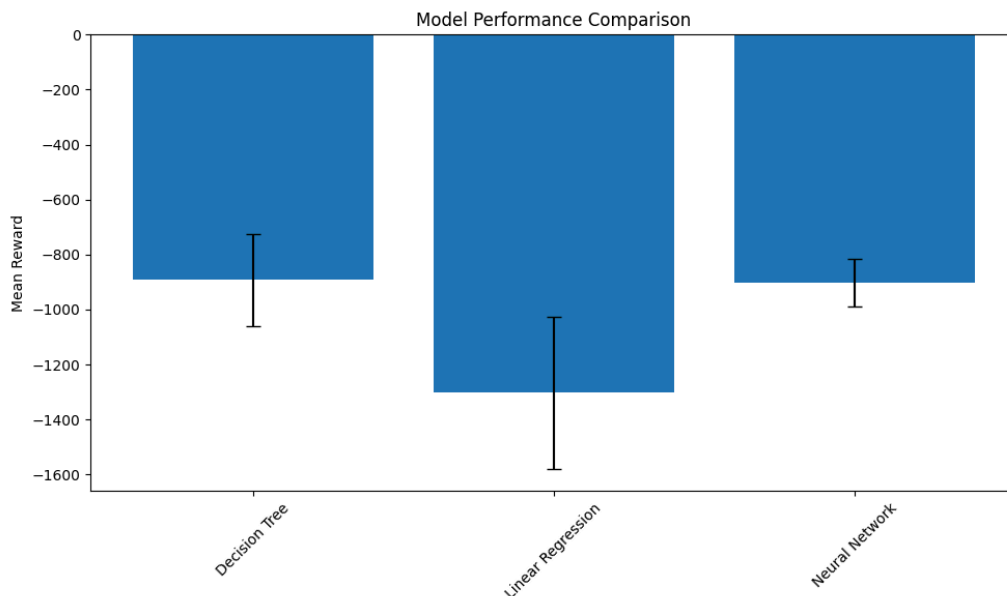
The summary of all of this data can be seen in the following table.

Metric	Early Stopping (True)	Early Stopping (False)	Difference (False - True)	Percentage Difference (%)
MSE	0,1853	0,1952	+0,0099	5.35%
RMSE	0,4305	0,4418	+0,0113	2.63%
MAE	0,1962	0,3356	+0,1394	71.05%

Exercise 4: Deploy the Agents

1. After selecting the best model for each approach to solve the problem it is time to deploy the agent. Modify the game loop for each of the three models and compare their performance. Now it is not possible to determine a goal. Select some other metrics to use as performance such as the accumulated reward. Are these models capable of solving the problem? If not, why not?

While they can't maintain the pendulum in total equilibrium all models try to solve the problem correctly.



The accumulated reward in this scenario is negative, meaning that closer values to 0 indicate better performance. Both the decision tree and the neural network demonstrate clear superiority. After running the script multiple times, the tree generally outperforms the neural network slightly. However, the tree's performance exhibits much higher variability, and in some instances, the neural network achieves better results within a smaller range.

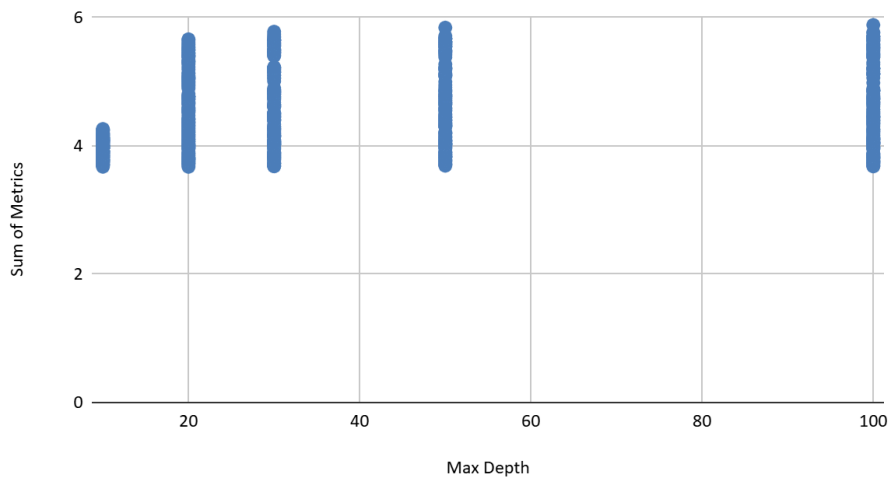
2. Does the best model (better test metrics) solve the problem the fastest? Why or why not? Why do some models perform better than others?

Metric-wise, the neural network is the best model. However, when analyzing the graphs and the data returned by the script, the decision tree appears to get closer to the goal. As the number of steps increases, the linear regression model continues to slow down, while the neural network speeds up.

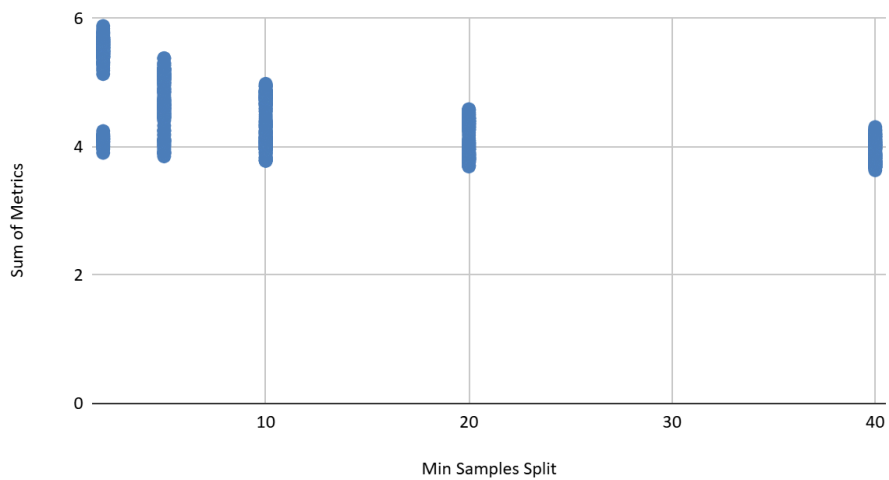
The tree's behavior, however, remains consistent. Since the lab specifies using 200 steps, the decision tree seems to perform best overall with this number of steps. The linear regression model, on the other hand, is the worst both in terms of metrics and results.

ANNEX (A collection of graphs to visit in case it's necessary):

Max Depth vs Sum of Metrics



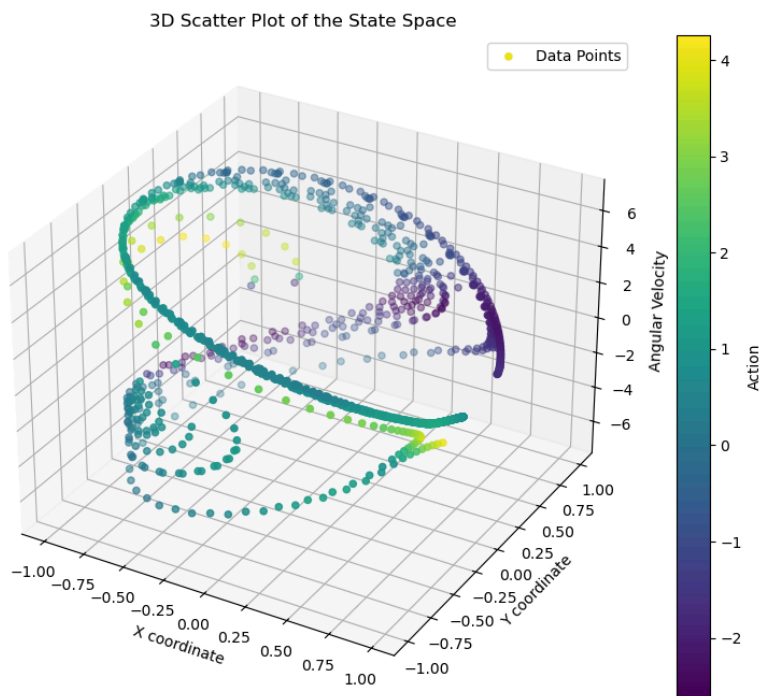
Min Samples Split vs Sum of Metrics



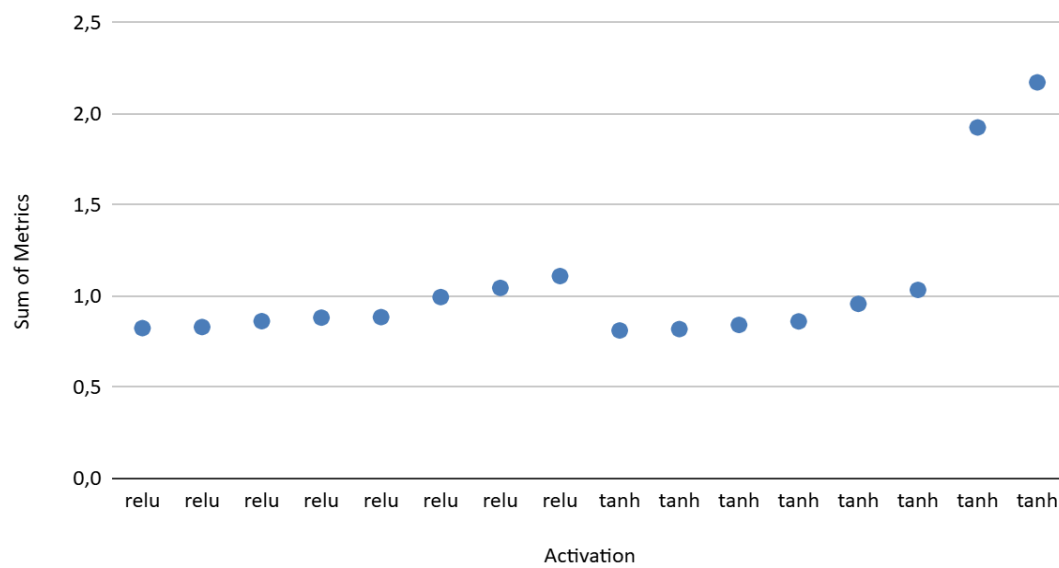
The chart displays the sum of metrics for various criteria. The y-axis represents the 'Sum of Metrics' from 0 to 6. The x-axis lists the 'Criterion' categories. The data is grouped into three main sections, each showing a general increase in the sum of metrics across the criteria.

Criterion	Sum of Metrics (approx.)
absolute_er...	3.8
absolute_er...	3.9
absolute_er...	4.0
absolute_er...	4.1
absolute_er...	4.2
absolute_er...	4.3
absolute_er...	4.4
absolute_er...	4.5
absolute_er...	4.6
absolute_er...	4.7
absolute_er...	4.8
absolute_er...	4.9
absolute_er...	5.0
absolute_er...	5.1
absolute_er...	5.2
absolute_er...	5.3
absolute_er...	5.4
absolute_er...	5.5
absolute_er...	5.6
absolute_er...	5.7
absolute_er...	5.8
absolute_er...	5.9
absolute_er...	6.0
absolute_er...	6.1
absolute_er...	6.2
absolute_er...	6.3
absolute_er...	6.4
absolute_er...	6.5
absolute_er...	6.6
absolute_er...	6.7
absolute_er...	6.8
absolute_er...	6.9
absolute_er...	7.0
absolute_er...	7.1
absolute_er...	7.2
absolute_er...	7.3
absolute_er...	7.4
absolute_er...	7.5
absolute_er...	7.6
absolute_er...	7.7
absolute_er...	7.8
absolute_er...	7.9
absolute_er...	8.0
absolute_er...	8.1
absolute_er...	8.2
absolute_er...	8.3
absolute_er...	8.4
absolute_er...	8.5
absolute_er...	8.6
absolute_er...	8.7
absolute_er...	8.8
absolute_er...	8.9
absolute_er...	9.0
absolute_er...	9.1
absolute_er...	9.2
absolute_er...	9.3
absolute_er...	9.4
absolute_er...	9.5
absolute_er...	9.6
absolute_er...	9.7
absolute_er...	9.8
absolute_er...	9.9
absolute_er...	10.0
absolute_er...	10.1
absolute_er...	10.2
absolute_er...	10.3
absolute_er...	10.4
absolute_er...	10.5
absolute_er...	10.6
absolute_er...	10.7
absolute_er...	10.8
absolute_er...	10.9
absolute_er...	11.0
absolute_er...	11.1
absolute_er...	11.2
absolute_er...	11.3
absolute_er...	11.4
absolute_er...	11.5
absolute_er...	11.6
absolute_er...	11.7
absolute_er...	11.8
absolute_er...	11.9
absolute_er...	12.0
absolute_er...	12.1
absolute_er...	12.2
absolute_er...	12.3
absolute_er...	12.4
absolute_er...	12.5
absolute_er...	12.6
absolute_er...	12.7
absolute_er...	12.8
absolute_er...	12.9
absolute_er...	13.0
absolute_er...	13.1
absolute_er...	13.2
absolute_er...	13.3
absolute_er...	13.4
absolute_er...	13.5
absolute_er...	13.6
absolute_er...	13.7
absolute_er...	13.8
absolute_er...	13.9
absolute_er...	14.0
absolute_er...	14.1
absolute_er...	14.2
absolute_er...	14.3
absolute_er...	14.4
absolute_er...	14.5
absolute_er...	14.6
absolute_er...	14.7
absolute_er...	14.8
absolute_er...	14.9
absolute_er...	15.0
absolute_er...	15.1
absolute_er...	15.2
absolute_er...	15.3
absolute_er...	15.4
absolute_er...	15.5
absolute_er...	15.6
absolute_er...	15.7
absolute_er...	15.8
absolute_er...	15.9
absolute_er...	16.0
absolute_er...	16.1
absolute_er...	16.2
absolute_er...	16.3
absolute_er...	16.4
absolute_er...	16.5
absolute_er...	16.6
absolute_er...	16.7
absolute_er...	16.8
absolute_er...	16.9
absolute_er...	17.0
absolute_er...	17.1
absolute_er...	17.2
absolute_er...	17.3
absolute_er...	17.4
absolute_er...	17.5
absolute_er...	17.6
absolute_er...	17.7
absolute_er...	17.8
absolute_er...	17.9
absolute_er...	18.0
absolute_er...	18.1
absolute_er...	18.2
absolute_er...	18.3
absolute_er...	18.4
absolute_er...	18.5
absolute_er...	18.6
absolute_er...	18.7
absolute_er...	18.8
absolute_er...	18.9
absolute_er...	19.0
absolute_er...	19.1
absolute_er...	19.2
absolute_er...	19.3
absolute_er...	19.4
absolute_er...	19.5
absolute_er...	19.6
absolute_er...	19.7
absolute_er...	19.8
absolute_er...	19.9
absolute_er...	20.0
absolute_er...	20.1
absolute_er...	20.2
absolute_er...	20.3
absolute_er...	20.4
absolute_er...	20.5
absolute_er...	20.6
absolute_er...	20.7
absolute_er...	20.8
absolute_er...	20.9
absolute_er...	21.0
absolute_er...	21.1
absolute_er...	21.2
absolute_er...	21.3
absolute_er...	21.4
absolute_er...	21.5
absolute_er...	21.6
absolute_er...	21.7
absolute_er...	21.8
absolute_er...	21.9
absolute_er...	22.0
absolute_er...	22.1
absolute_er...	22.2
absolute_er...	22.3
absolute_er...	22.4
absolute_er...	22.5
absolute_er...	22.6
absolute_er...	

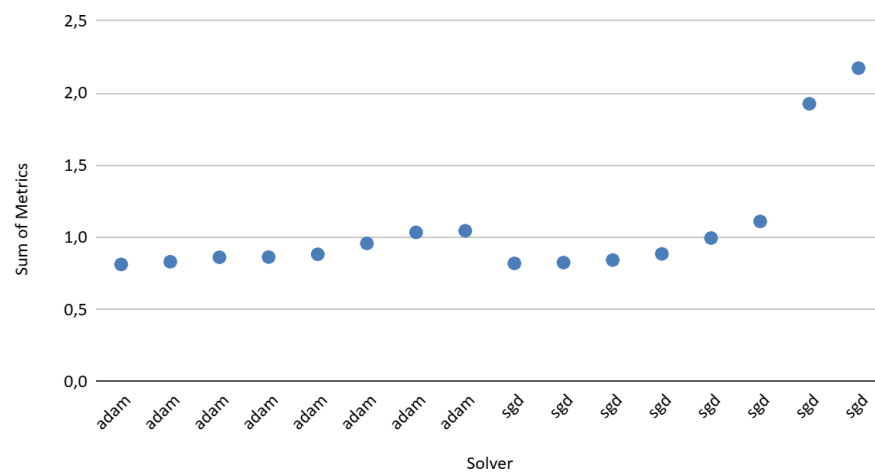
[illegible]



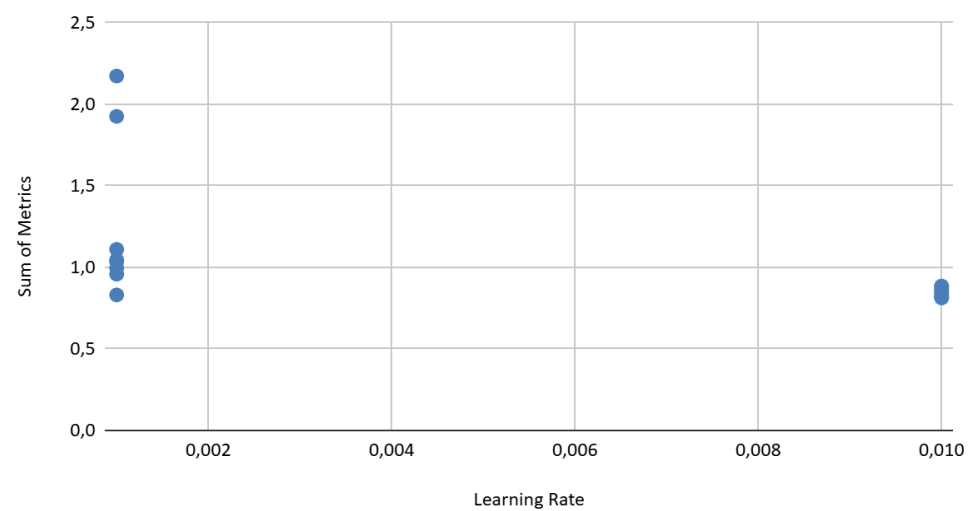
Sum of Metrics frente a Activation



Sum of Metrics frente a Solver



Sum of Metrics frente a Learning Rate



Hidden Layer Size vs Sum of Metrics

